

16-BIT ARITHMETIC LOGIC UNIT (ALU) DESIGN USING VERILOG HDL

Project Documentation

Submitted By

Name: Jaivignesh M

Department: Electronics and Communication Engineering

Institution: University College of Engineering Kanchipuram

University: Anna University

ABSTRACT

An Arithmetic Logic Unit (ALU) is one of the fundamental building blocks of a processor responsible for executing arithmetic and logical operations. This project presents the design and implementation of a **16-bit ALU using Verilog HDL**. The ALU performs sixteen different operations selected through a 4-bit opcode. Arithmetic operations include addition, subtraction, increment, and decrement. Logical operations include AND, OR, XOR, NOT, NAND, NOR, and XNOR. It also supports shift operations and comparison functions.

The design is simulated and verified using **Xilinx Vivado**, and the RTL schematic is generated through synthesis. The project demonstrates digital design concepts, combinational logic implementation, and functional verification using a Verilog testbench.

TABLE OF CONTENTS

1. Introduction
2. Objectives
3. Problem Statement
4. Software Requirements
5. Hardware Requirements
6. ALU Architecture
7. Block Diagram
8. Internal Architecture

9. Opcode Table
10. Verilog Design
11. Testbench
12. Simulation Results
13. RTL Schematic
14. Advantages
15. Applications
16. Future Scope
17. Conclusion
18. References

1. INTRODUCTION

The Arithmetic Logic Unit (ALU) is the computational core of every processor. It performs arithmetic calculations, logical operations, comparisons, and bit manipulation. The performance of a processor depends significantly on the efficiency of its ALU.

This project implements a 16-bit ALU using Verilog HDL. The design follows a combinational logic approach, where the output changes immediately based on the input values and opcode.

2. OBJECTIVES

- Design a 16-bit ALU using Verilog HDL.
 - Implement arithmetic and logical operations.
 - Generate status flags.
 - Verify the design using a testbench.
 - Synthesize the design in Vivado.
 - Generate RTL schematic.
-

3. PROBLEM STATEMENT

To design and implement a 16-bit Arithmetic Logic Unit capable of performing multiple arithmetic and logical operations based on a 4-bit control signal while generating appropriate status flags.

4. SOFTWARE REQUIREMENTS

Software	Version
Xilinx Vivado	2023/2024
Verilog HDL	IEEE 1364
Windows 10/11 64-bit	

5. HARDWARE REQUIREMENTS

- AMD RYZEN i3/i5/i7 Processor
 - Minimum 4 GB RAM (8 GB Recommended)
 - 10 GB Free Disk Space
 - FPGA Board (Optional)
-

6. ALU ARCHITECTURE

The ALU receives two 16-bit operands (**A** and **B**) along with a 4-bit opcode. Depending on the opcode, it performs one of sixteen operations and produces a 16-bit result with status flags.

Inputs

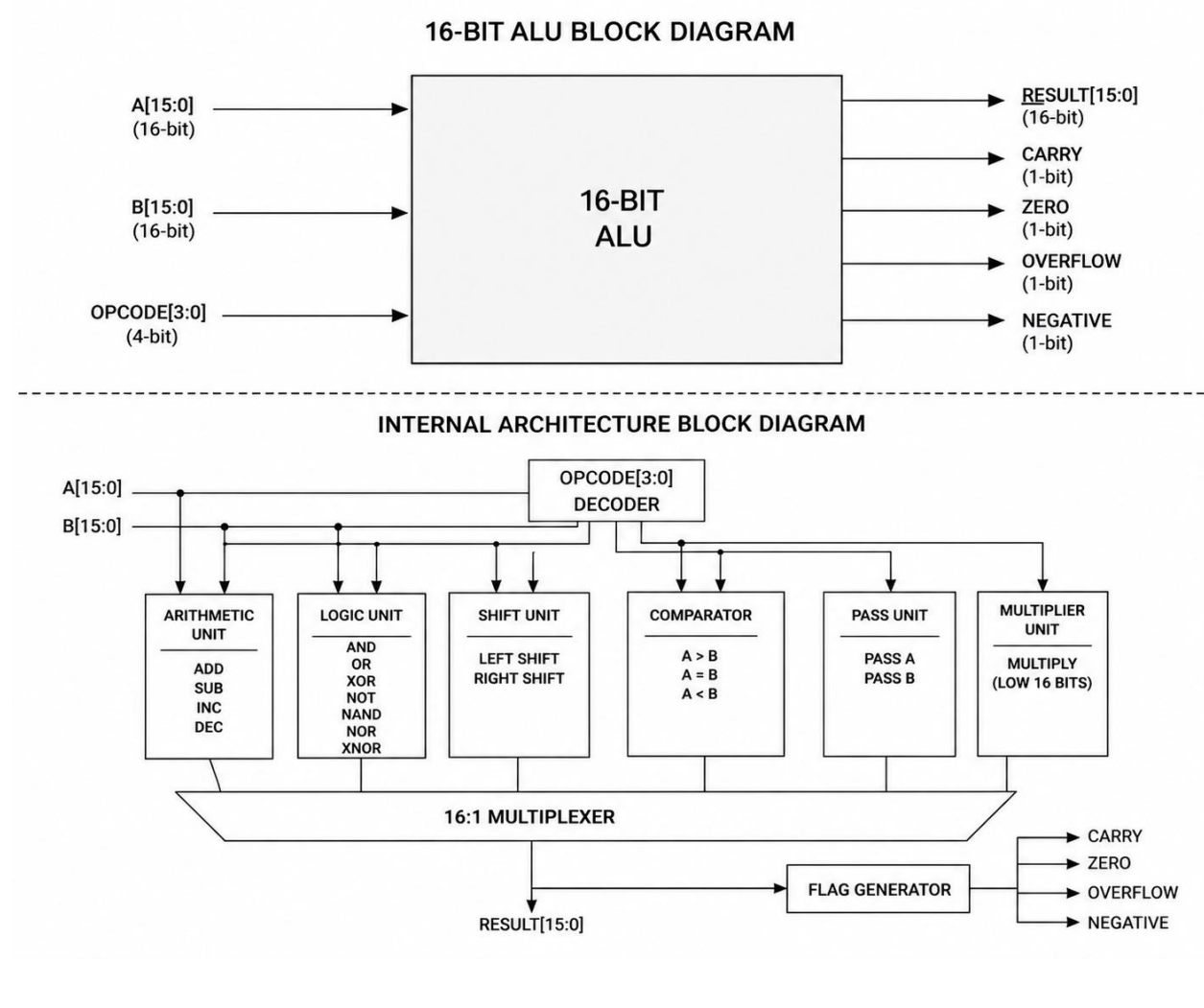
- A[15:0]
- B[15:0]
- Opcode[3:0]

Outputs

- Result[15:0]
- Carry

- Zero
- Overflow
- Negative

7. BLOCK DIAGRAM



8. INTERNAL ARCHITECTURE

The ALU consists of the following blocks:

- Opcode Decoder
 - Arithmetic Unit
 - Logic Unit
 - Shift Unit
 - Comparator
 - Multiplexer
 - Flag Generator
-

9. OPCODE TABLE

Opcode Operation

0000	Addition
0001	Subtraction
0010	AND
0011	OR
0100	XOR
0101	NOT
0110	Shift Left
0111	Shift Right
1000	Increment
1001	Decrement
1010	Compare
1011	NAND

Opcode Operation

1100	NOR
1101	XNOR
1110	Pass A
1111	Pass B

10. DESIGN METHODOLOGY

The ALU is designed using a combinational always @(*) block. A case statement selects the required operation based on the opcode.

The status flags are generated after every operation:

- Carry Flag
 - Zero Flag
 - Overflow Flag
 - Negative Flag
-

11. VERILOG DESIGN

The Verilog module includes:

- Module declaration
 - Input and output ports
 - Temporary register
 - Case statement
 - Flag generation logic
-

12. TESTBENCH

The verification process includes testing all 16 operations with different input combinations.

Simulation steps:

- Apply inputs A and B.
- Change opcode.
- Observe output.
- Verify status flags.

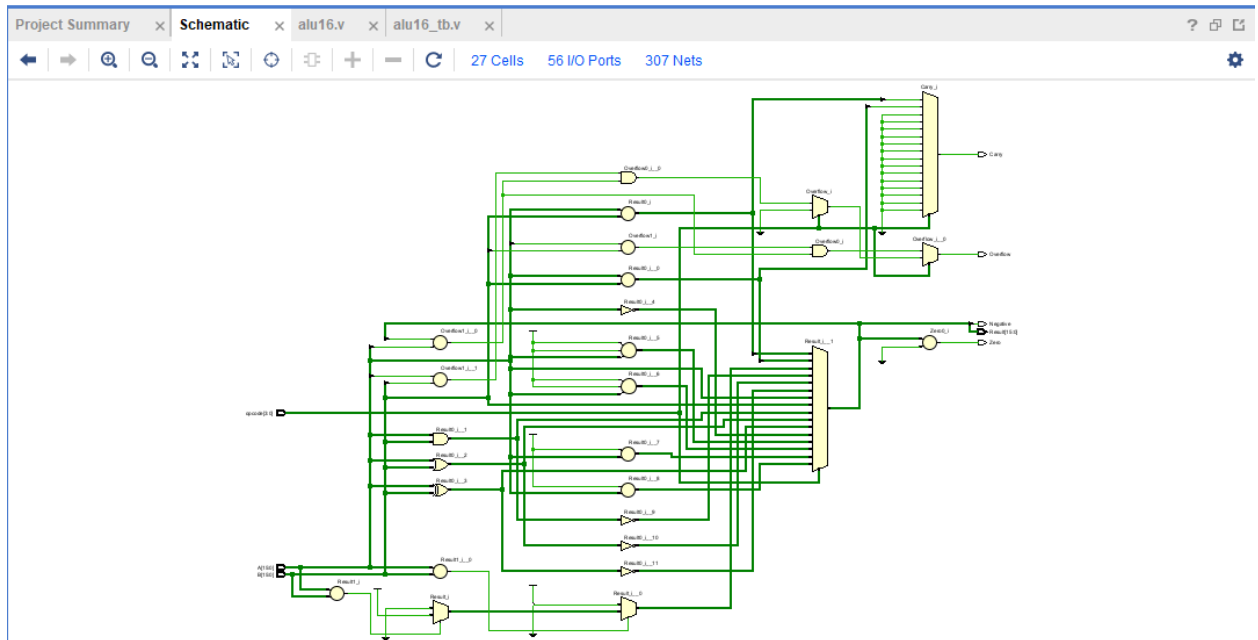
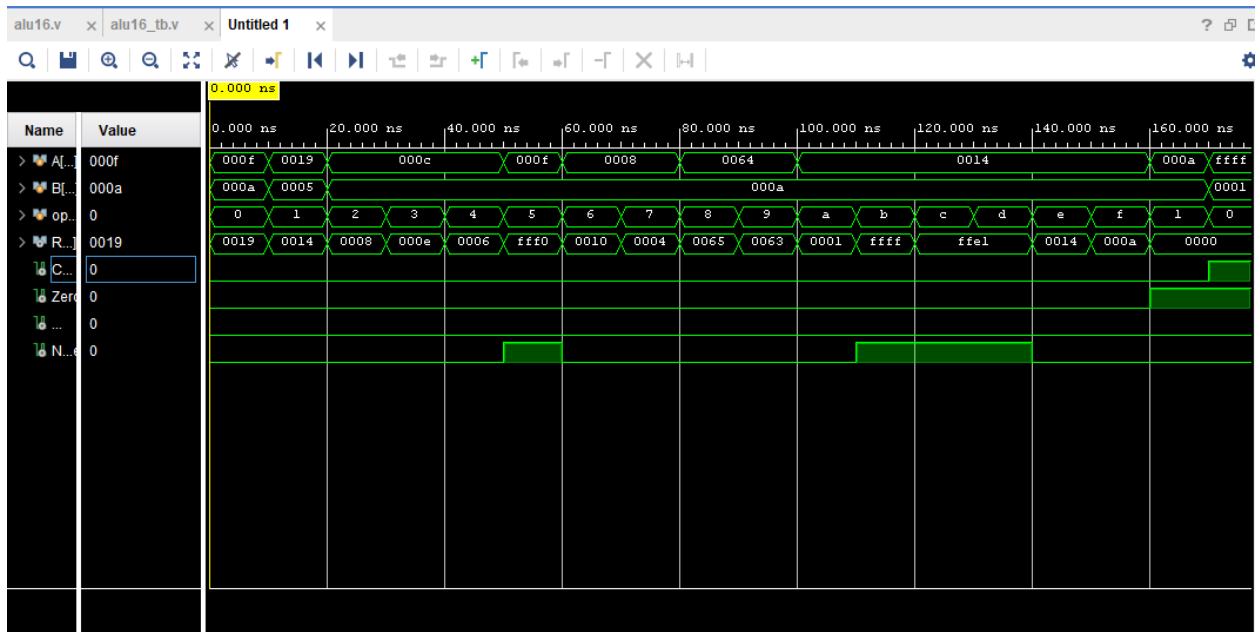
(Paste the complete alu16_tb.v code here.)

13. SIMULATION RESULTS

The simulation verifies all ALU operations successfully.

Sample Results

A	B	Opcode	Operation	Result
15	10 0000		Addition	25
25	5 0001		Subtraction	20
12	10 0010	AND		8
12	10 0011	OR		14
12	10 0100	XOR		6
15	0 0101	NOT		65520
8	0 0110	Shift Left		16
8	0 0111	Shift Right		4
100	0 1000	Increment		101
100	0 1001	Decrement		99
20	10 1010	Compare		1



14. STATUS FLAGS

Carry Flag

Indicates carry generation during addition.

Zero Flag

Becomes 1 when the result is zero.

Overflow Flag

Indicates signed arithmetic overflow.

Negative Flag

Reflects the MSB of the result.

15. ADVANTAGES

- High-speed combinational design
 - Easy FPGA implementation
 - Modular and scalable
 - Simple verification
 - Suitable for processor design
 - Low hardware complexity
-

16. APPLICATIONS

- CPUs
- Microprocessors
- Microcontrollers
- DSP systems
- Embedded systems
- FPGA-based systems
- Digital signal processing

- Computer architecture laboratories
-

17. FUTURE SCOPE

- 32-bit ALU
 - 64-bit ALU
 - Floating-point ALU
 - Barrel shifter
 - Multiply-Accumulate (MAC) Unit
 - Pipeline architecture
 - RISC-V processor integration
-

18. CONCLUSION

A 16-bit Arithmetic Logic Unit was successfully designed and implemented using Verilog HDL. The ALU performs arithmetic, logical, shift, and comparison operations selected by a 4-bit opcode. Functional verification using a Verilog testbench confirmed correct operation, and the design is synthesizable in Xilinx Vivado. This project strengthens understanding of RTL design, combinational logic, and digital system implementation, making it an excellent foundation for more advanced processor and FPGA designs.

REFERENCES

1. M. Morris Mano, *Digital Design*, 6th Edition, Pearson.
 2. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition.
 3. Stephen Brown & Zvonko Vranesic, *Fundamentals of Digital Logic with Verilog Design*.
 4. Xilinx, *Vivado Design Suite User Guide*.
 5. IEEE Standard 1364 – Verilog HDL.
-